Tab 5

(1a) Program:

```
#include <stdio.h>
#define MAX_SIZE 100 // Maximum size of the array void createArray(int arr[], int *size);
void insertElement(int arr[], int *size, int element, int position);
int searchElement(int arr[], int size, int element);
void deleteElement(int arr[], int *size, int position);
 void displayArray(int arr[], int size);
int main() {
int arr[MAX_SIZE];
int size = 0;
int choice, element, position, result;
 while(1) {
printf("\nArray Operations Menu:\n");
printf("1. Create Array\n");
printf("2. Insert Element\n");
printf("3. Search Element\n");
printf("4. Delete Element\n"); printf("5.DisplayArray\n");
printf("6. Exit\n");
 printf("Enter your choice: ");
scanf("%d", &choice);
switch(choice) {
case 1:
createArray(arr, &size); break;
case 2:
printf("Enter element to insert: ");
 scanf("%d", &element);
printf("Enter position to insert (0 to %d): ", size); scanf("%d", &position);
insertElement(arr, &size, element, position); break;
case 3:
printf("Enter element to search: ");
 scanf("%d", &element);
result = searchElement(arr, size, element); if(result != -1)
printf("Element found at position: %d\n", result); else
printf("Element not found in the array.\n"); break;
case 4:
printf("Enter position to delete (0 to %d): ", size - 1); scanf("%d", &position);
deleteElement(arr, &size, position);
break;
case 5:
displayArray(arr, size); break;
case 6:
return 0;
 default:
```

```
printf("Invalid choice! Please enter a valid option.\n");
}
}
return 0;
}
void createArray(int arr[], int *size) { int n, i;
printf("Enter the number of elements: "); scanf("%d", &n);
if(n > MAX_SIZE) {
printf("Error: Number of elements exceeds the maximum array size.\n");return;
}
printf("Enter %d elements: ", n);
for(i = 0; i < n; i++)
{ scanf("%d", &arr[i]);
}
*size = n;
printf("Array created successfully.\n");
}
void insertElement(int arr[], int *size, int element, int position) {
if(*size >= MAX_SIZE) {

printf("Error: Array is full. Cannot insert element.\n");
return;
}
if(position < 0 || position > *size) {
 printf("Error: Invalid position.\n"); return;
}
for(int i = *size; i > position; i--) {
 arr[i] = arr[i - 1];
}
arr[position] = element; (*size)++;
printf("Element inserted successfully.\n");
}
int searchElement(int arr[], int size, int element) { for(int i = 0; i < size; i++) {
if(arr[i] == element) { return i;
}
}
return -1;
}
void deleteElement(int arr[], int *size, int position) { if(position < 0 || position >= *size) {
printf("Error: Invalid position.\n"); return;
}
for(int i = position; i < *size - 1; i++) {

printf("Array is empty.\n");
```

```
 return;
}
printf("Array elements: ");
 for(int i = 0; i < size; i++) {
printf("%d ", arr[i]);
}
printf("\n");
}arr[i] = arr[i + 1];
}
(*size)--;
printf("Element deleted successfully.\n");
}

void displayArray(int arr[], int size) { if(size == 0) {
printf("Array is empty.\n");
return;
}
printf("Array elements: ");
for(int i = 0; i < size; i++) {
printf("%d ", arr[i]);
}
printf("\n");
}
```

Output

Array Operations Menu:
1.Create Array
2.Insert Element
3.Search Element
4.Delete Element
5.Display Array
6.Exit
Enter your choice: 1
Enter the number of elements: 5 Enter 5 elements: 1 2 3 4 5 Array created successfully.

Array Operations Menu:
1.Create Array
2.Insert Element
3.Search Element
4.Delete Element
5.Display Array
6.Exit
Enter your choice: 2

Enter element to insert: 10
Enter position to insert (0 to 5): 2 Element inserted successfully.

Array Operations Menu:
1.reate Array
2.Insert Element
3.Search Element
4.Delete Element
5.Display Array
6.Exit

Enter your choice: 5
Array elements: 1 2 10 3 4 5

Array Operations Menu:
1.Create Array
2.Insert Element
3.Search Element
4.Delete Element
5.Display Array
6.ExitEnter element to search: 10 Element found at position: 2

Array Operations Menu:
Create Array
Insert Element
Search Element
Delete Element
Display Array
Exit
Enter your choice: 4
Enter position to delete (0 to 5): 2 Element deleted successfully.
Array Operations Menu:
1.reate Array
2.Insert Element
3.Search Element
4.Delete Element
5.Display Array
6.Exit
Enter your choice: 5 Array elements: 1 2 3 4 5

Tab 3

(1b) program

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
int data;
struct Node* next;
};
struct Node* head = NULL;
// Function to create a new node struct Node* createNode(int data) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node)); newNode->data = data;
newNode->next = NULL; return newNode;
}
// Function to insert a node at the beginning void insertAtBeginning(int data) {
struct Node* newNode = createNode(data); newNode->next = head;
head = newNode;
}
// Function to insert a node at the end void insertAtEnd(int data) {
struct Node* newNode = createNode(data); if (head == NULL) {
head = newNode; return;
}
struct Node* temp = head; while (temp->next != NULL) { temp = temp->next;
}
temp->next = newNode;
}
// Function to insert a node at a specified position void insertAtPosition(int data, int position) {
if (position < 1) { printf("Invalid position\n"); return;
}
if (position == 1) { insertAtBeginning(data); return;
}
struct Node* newNode = createNode(data); struct Node* temp = head;
int count = 1;
while (count < position - 1 && temp != NULL) { temp = temp->next;
count++;
}
if (temp == NULL) { printf("Invalid position\n"); return;
}

newNode->next = temp->next; temp->next = newNode;
}

// Function to delete a node from the beginning void deleteAtBeginning() {
if (head == NULL) { printf("List is empty\n"); return;
}
```

```
struct Node* temp = head; head = head->next; free(temp);
}
// Function to delete a node from the end void deleteAtEnd() {


if (head == NULL) { printf("List is empty\n"); return;
}
if (head->next == NULL) { free(head);
head = NULL; return;
}
struct Node* temp = head;
while (temp->next->next != NULL) { temp = temp->next;
}
free(temp->next); temp->next = NULL;
}
// Function to delete a node at a specified position void deleteAtPosition(int position) {
if (position < 1) { printf("Invalid position\n"); return;
}
if (position == 1) { deleteAtBeginning(); return;
}
struct Node* temp = head; int count = 1;
while (count < position - 1 && temp != NULL && temp->next != NULL) { temp = temp->next;
count++;
}
if (temp == NULL || temp->next == NULL) { printf("Invalid position\n");
return;
}
struct Node* toDelete = temp->next; temp->next = toDelete->next; free(toDelete);
}
// Function to display the linked list void display() {
if (head == NULL) { printf("List is empty\n"); return;
}
struct Node* temp = head; while (temp != NULL) { printf("%d -> ", temp->data); temp =
temp->next;
}


printf("NULL\n");
}
int main() {
int choice, data, position;

while (1) {
printf("\nSingly Linked List Operations:\n"); printf("1. Insert at Beginning\n");
```

```
printf("2. Insert at End\n");
printf("3. Insert at Position\n");
printf("4. Delete at Beginning\n");
printf("5. Delete at End\n");
printf("6. Delete at Position\n");
 printf("7. Display\n");
printf("8. Exit\n");
printf("Enter your choice: ");
 scanf("%d", &choice);

switch (choice) { case 1:
printf("Enter data to insert at beginning: "); scanf("%d", &data); insertAtBeginning(data);
break; case 2:
printf("Enter data to insert at end: "); scanf("%d", &data); insertAtEnd(data);
break; case 3:
printf("Enter data to insert: "); scanf("%d", &data); printf("Enter position: "); scanf("%d",
&position); insertAtPosition(data, position); break;
case 4: deleteAtBeginning(); break;
case 5: deleteAtEnd(); break;
case 6:
printf("Enter position to delete: ");
 scanf("%d", &position); deleteAtPosition(position);
break;
case 7:
display();
break;


case 8:
exit(0);
 default:
printf("Invalid choice\n");
}
}
return 0;
}
```

Output

```
Singly Linked List Operations:
1.Insert at Beginning
2.Insert at End
3.Insert at Position
```

4.Delete at Beginning
5.Delete at End
6.Delete at Position
7.Display
8.Exit

Enter your choice: 1
Enter data to insert at beginning: 1

Singly Linked List Operations:
1.Insert at Beginning
2.Insert at End
3.Insert at Position
4.Delete at Beginning
5.Delete at End
6.Delete at Position
7.Display
8.Exit

Enter your choice: 1
Enter data to insert at beginning: 2

Singly Linked List Operations:
1.Insert at Beginning
2.Insert at End
3.Insert at Position
4.Delete at Beginning
5.Delete at End
6.Delete at Position
7.Display
8.Exit

Enter your choice: 7 2 -> 1 -> NULL

Singly Linked List Operations:
1.Insert at Beginning
2.Insert at End
3.Insert at Position
4.Delete at Beginning
5.Delete at End
6.Delete at Position
7.Display
8.Exit

Enter your choice: 2
Enter data to insert at end: 3

Singly Linked List Operations:
1.Insert at Beginning
2.Insert at End
3.Insert at Position
4.Delete at Beginning
5.Delete at End
6.Delete at Position
7.Display
8.Exit

Enter your choice: 2
Enter data to insert at end: 4

Singly Linked List Operations:
1.Insert at Beginning
2.Insert at End
3.Insert at Position
4.Delete at Beginning
5.Delete at End
6.Delete at Position
7.Display
8.Exit

Enter your choice: 7
2 -> 1 -> 3 -> 4 -> NULL

Singly Linked List Operations:
1.Insert at Beginning
2.Insert at End
3.Insert at Position
4.Delete at Beginning
5.Delete at End
6.Delete at Position
7.Display
8.Exit

Enter your choice: 3 Enter data to insert: 5 Enter position: 3

Singly Linked List Operations:
1.Insert at Beginning
2.Insert at End

3.Insert at Position
4.Delete at Beginning
5.Delete at End
6.Delete at Position
7.Display
8.Exit

Enter your choice: 7
2 -> 1 -> 5 -> 3 -> 4 -> NULL

Singly Linked List Operations:
1.Insert at Beginning
2.Insert at End
3.Insert at Position
4.Delete at Beginning
5.Delete at End
6.Delete at Position
7.Display
8.Exit

Enter your choice: 4

Singly Linked List Operations:
N
1.Insert at Beginning
2.Insert at End
3.Insert at Position
4.Delete at Beginning
5.Delete at End
6.Delete at Position
7.Display
8.Exit

Enter your choice: 7
1 -> 5 -> 3 -> 4 -> NULL

Singly Linked List Operations:
1.Insert at Beginning
2.Insert at End
3.Insert at Position
4.Delete at Beginning
5.Delete at End
6.Delete at Position
7.Display

8.Exit

Enter your choice: 5

Singly Linked List Operations:
1.Insert at Beginning
2.Insert at End
3.Insert at Position
4.Delete at Beginning
5.Delete at End
6.Delete at Position
7.Display
8.Exit

Enter your choice: 7 1 -> 5 -> 3 -> NULL

Singly Linked List Operations:
Enter your choice: 6 Enter position to delete: 3
Singly Linked List Operations:
1.Insert at Beginning
2.Insert at End
3.Insert at Position
4.Delete at Beginning
5.Delete at End
6.Delete at Position
7.Display
8.Exit

Enter your choice: 7 1 -> 5 -> NULL

Singly Linked List Operations:
1.Insert at Beginning
2.Insert at End
3.Insert at Position
4.Delete at Beginning
5.Delete at End
6.Delete at Position
7.Display
8.Exit

Enter your choice: 8

Tab 4

2) program

```
#include <stdio.h>
 #include <stdlib.h>
 struct node {
int data;
struct node *next;
};
struct node *head = NULL;
// Function to create a new node struct node* createNode(int data) {
struct node *newNode = (struct node*)malloc(sizeof(struct node)); newNode->data = data;
newNode->next = NULL; return newNode;
}
// Function to create a circular linked list void createList(int data) {
head = createNode(data);
head->next = head; // Make it circular
}
// Function to insert a node at the beginning void insertAtBeginning(int data) {
struct node *newNode = createNode(data);
 if (head == NULL) {
head = newNode; head->next = head;
} else {
newNode->next = head; struct node *temp = head; while (temp->next != head) { temp =
temp->next;
}
temp->next = newNode; head = newNode;
}}
// Function to insert a node at the end void insertAtEnd(int data) {
struct node *newNode = createNode(data); if (head == NULL) {
head = newNode;
head->next = head;
} else {
struct node *temp = head;
 while (temp->next != head) { temp = temp->next;
}
temp->next = newNode;
newNode->next = head;
}
}
// Function to insert a node at a specified position void insertAtPosition(int data, int position) {
if (position < 1) {
printf("Invalid position\n");
return;
}
```

```
if (position == 1) { insertAtBeginning(data); return;
}
struct node *newNode = createNode(data); struct node *temp = head;
int count = 1;
while (count < position - 1 && temp->next != head) { temp = temp->next;
count++;
}

if (count == position - 1) { newNode->next = temp->next; temp->next = newNode;
} else {
printf("Invalid position\n");
}
}
// Function to delete a node from the beginning void deleteAtBeginning() {
if (head == NULL) {
 printf("List is empty\n"); return;
}

if (head->next == head) { free(head);
head = NULL;
} else {
struct node *temp = head; head = head->next;
struct node *last = head;
while (last->next != temp) {
 last = last->next;
}
last->next = head;
 free(temp);
}
}

// Function to delete a node from the end void deleteAtEnd() {


if (head == NULL) {
 printf("List is empty\n");
return;
}
if (head->next == head) {
 free(head);
head = NULL;
} else {
struct node *temp = head;
while (temp->next->next != head) {
```

```
 temp = temp->next;
}
free(temp->next);
 temp->next = head;
}
}

// Function to delete a node at a specified position void deleteAtPosition(int position) {
if (position < 1) {
printf("Invalid position\n");
return;
}

if (position == 1) {
 deleteAtBeginning();
return;
}
struct node *temp = head;
 int count = 1;
while (count < position - 1 && temp->next != head) {
 temp = temp->next;
count++;
}

if (count == position - 1) {
struct node *toDelete = temp->next;
 temp->next = toDelete->next;
 free(toDelete);
}
 else {
printf("Invalid position\n");
}
}

// Function to display the circular linked list void display() {


if (head == NULL) {
printf("List is empty\n"); return;
}
struct node *temp = head;
 do {
printf("%d ", temp->data); temp = temp->next;
} while (temp != head);
```

```
printf("\n");
}
int main() {
int choice, data, position;
 while (1) {
printf("\nCircular Linked List Operations:\n"); printf("1. Create List\n");
printf("2. Insert at Beginning\n");
printf("3. Insert at End\n");
printf("4. Insert at Position\n");
printf("5. Delete at Beginning\n");
printf("6. Delete at End\n");
printf("7. Delete at Position\n");
printf("8. Display\n");
printf("9. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
 switch (choice) {
case 1:
printf("Enter data for the first node: "); scanf("%d", &data);
createList(data);
 break;
case 2:
printf("Enter data to insert at beginning: "); scanf("%d", &data);
 insertAtBeginning(data);
break;
case 3:
printf("Enter data to insert at end: "); scanf("%d", &data); insertAtEnd(data);
break;
case 4:
printf("Enter data to insert: ");
 scanf("%d", &data);
printf("Enter position: ");
 scanf("%d", &position);

insertAtPosition(data, position);
 break;
case 5: deleteAtBeginning();
 break;
case 6: deleteAtEnd();
break;
case 7:
printf("Enter position to delete: ");
 scanf("%d", &position); deleteAtPosition(position);
break;
```

```
case 8:
display();
break;
case 9:
exit(0);
default:
printf("Invalid choice\n");
}
}
return 0;
}
```

Output

```
Circular Linked List Operations:
1.Create List
2.Insert at Beginning
3.Insert at End
4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position
8.Display
9.Exit
Enter your choice: 2
Enter data to insert at beginning: 1

Circular Linked List Operations:
1.Create List
2.Insert at Beginning
3.Insert at End
4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position
8.Display
9.Exit
Enter your choice: 2
Enter data to insert at beginning: 2

Circular Linked List Operations:
1.Create List
```

2.Insert at Beginning
3.Insert at End
4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position
8.Display
9.Exit
Enter your choice: 8 2 1

Circular Linked List Operations:
1.Create List
2.Insert at Beginning
3.Insert at End
4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position
8.Display
9.Exit
Enter your choice: 3
Enter data to insert at end: 3

Circular Linked List Operations:
1.Create List
2.Insert at Beginning
3.Insert at End
4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position
8.Display
9.Exit
Enter your choice: 3
Enter data to insert at end: 4

Circular Linked List Operations:
1.Create List
2.Insert at Beginning
3.Insert at End
4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position

8.Display
9.Exit
Enter your choice: 8 2 1 3 4

Circular Linked List Operations:
D1.Create List
2.Insert at Beginning
3.Insert at End
4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position
8.Display
9.Exit
Enter your choice: 4 Enter data to insert: 5 Enter position: 3

Circular Linked List Operations:
1.Create List
2.Insert at Beginning
3.Insert at End
4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position
8.Display
9.Exit
Enter your choice: 8 2 1 5 3 4

Circular Linked List Operations:
1.Create List
2.Insert at Beginning
3.Insert at End
4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position
8.Display
9.Exit
Enter your choice: 5

Circular Linked List Operations:
1.Create List
2.Insert at Beginning
3.Insert at End

4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position
8.Display
9.Exit
Enter your choice: 8 1 5 3 4

Circular Linked List Operations:
1.Create List
2.Insert at Beginning
3.Insert at End
4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position
8.Display
9.Exit
Enter your choice: 6

Circular Linked List Operations:
1.Create List
2.Insert at Beginning
3.Insert at End
4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position
8.Display
9.Exit
Enter your choice: 8 1 5 3

Circular Linked List Operations:
1.Create List
2.Insert at Beginning
3.Insert at End
4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position
8.Display
9.Exit
Enter your choice: 7 Enter position to delete: 2

Circular Linked List Operations:
1.Create List
2.Insert at Beginning
3.Insert at End
4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position
8.Display
9.Exit

Enter your choice: 8 1 3

Circular Linked List Operations:
1.Create List
2.Insert at Beginning
3.Insert at End
4.Insert at Position
5.Delete at Beginning
6.Delete at End
7.Delete at Position
8.Display
9.Exit